



Designing Artificial Intelligence Products

Week 3 Office Hour
11 Sep 2026

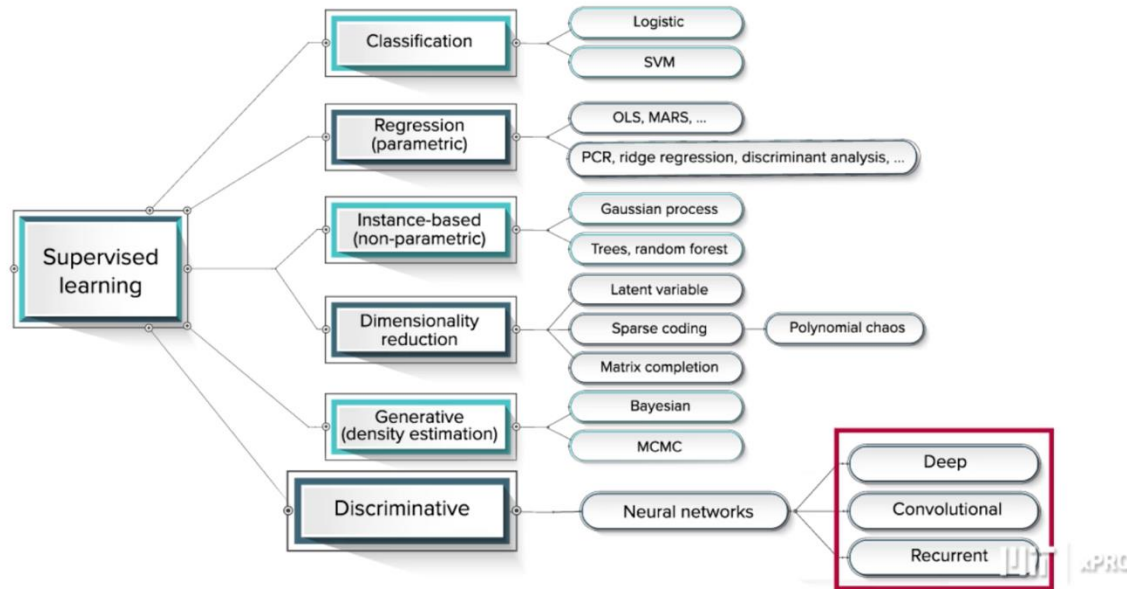
Danny Malter



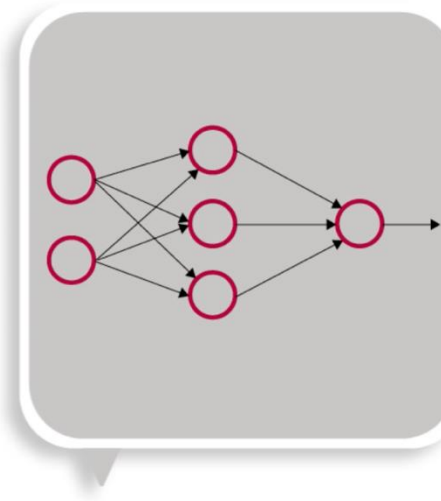
Review Week 3

Video 1: Introduction to Deep Learning

Introduction to Deep Learning



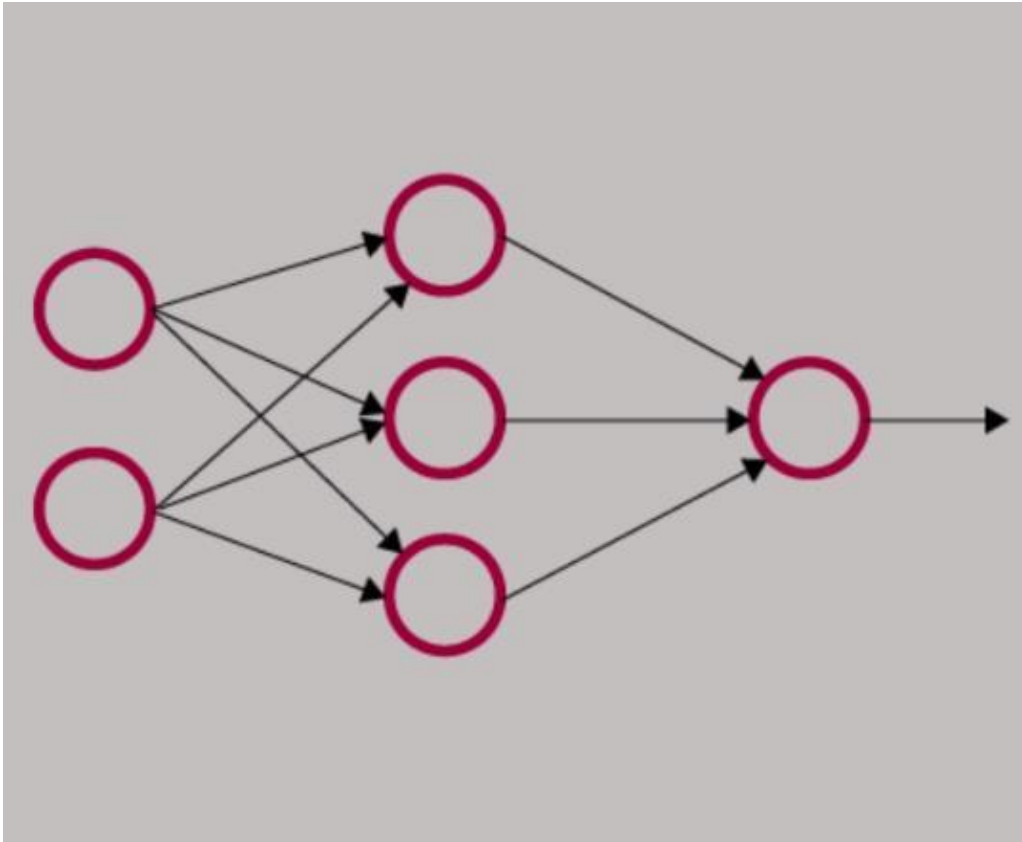
Machine Learning Methods



Neural networks

- Multi-layer perceptrons (MLPs)
- Deep neural networks
 - Convolutional neural networks (CNNs)
 - Recurrent neural networks (RNNs)

Introduction to Deep Learning



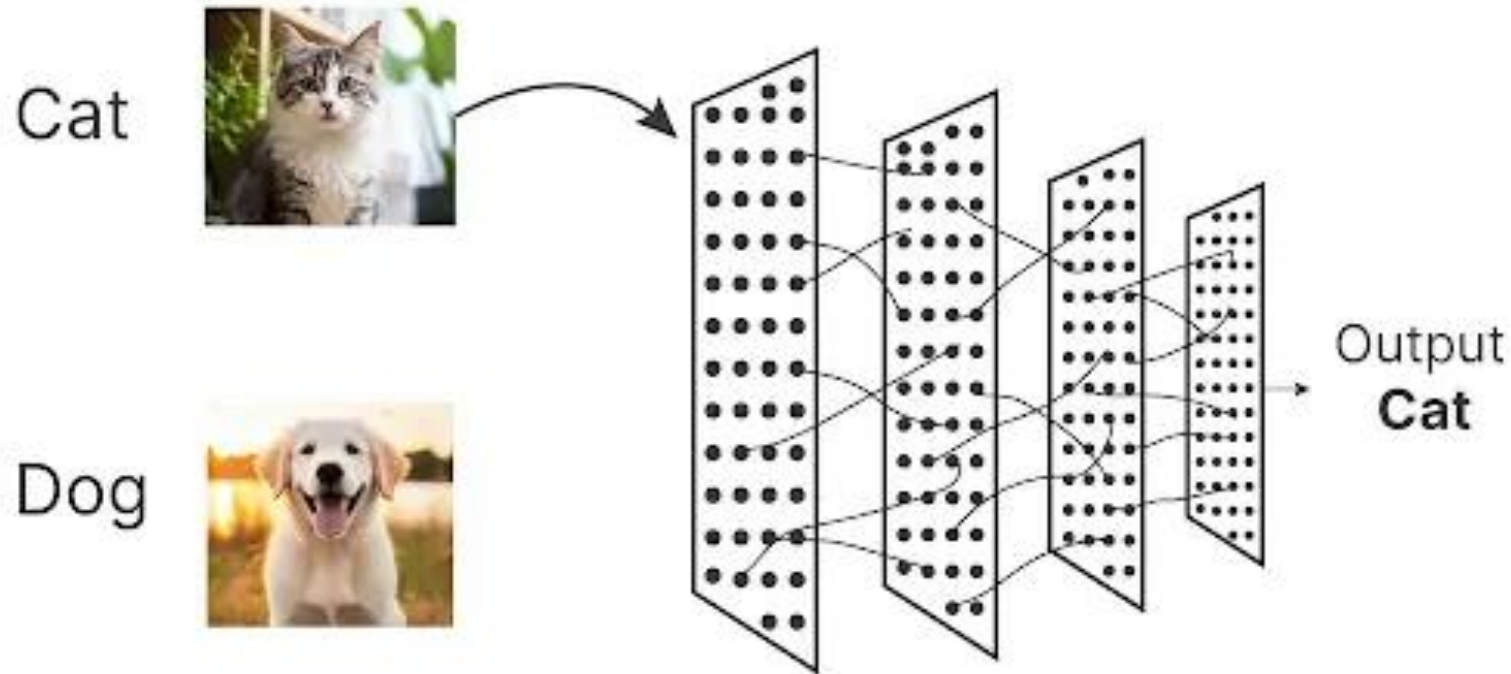
- **Multi-Layer Perceptrons (MLPs)**
 - Classical fully-connected networks; foundation of modern deep learning
 - Use cases: tabular data, classification, regression
- **Convolutional Neural Networks (CNNs)**
 - Use spatial filters to detect local patterns (edges, textures, shapes)
 - Use cases: image classification, object detection, medical imaging
- **Recurrent Neural Networks (RNNs / LSTMs)**
 - Process sequences by maintaining hidden state across time steps
 - Use cases: speech, time-series, stock prediction, early NLP
- **Transformers**
 - Self-attention captures long-range dependencies in parallel
 - Use cases: LLMs (GPT, Claude), translation, vision, multimodal AI

Convolutional Neural Network – Image Recognition

Image Classification

Assigning a label to an image from a fixed set of categories.

- **Input:** raw pixel values (e.g., $224 \times 224 \times 3$)
- **Output:** class probabilities (softmax)
- **Why CNNs?** learn spatial features via convolution & pooling



Real-World Classification Tasks Medical imaging (tumor vs. healthy) | Quality control (defect detection) | Content moderation | Autonomous vehicles (sign recognition) | Face ID | Wildlife / plant species ID

Key challenges: variations in lighting, scale, viewpoint, occlusion, and class imbalance

CNN vs. Multimodal Vision Model (Llama 4 Maverick)

Scenario: An insurance company receives a photo of a damaged vehicle and needs to process the claim.

Traditional CNN

What it does

- Trained for one specific task (e.g., classify damage type)
- Outputs a label or bounding box — nothing more

Example output

- "Front bumper damage" — 92% confidence
- "Headlight damage" — 87% confidence

Strengths

- Fast, cheap, runs on edge devices
- Highly accurate on its narrow task

Limitations

- Cannot reason or explain
- Needs labeled training data per use case
- One model = one task

Multimodal (Meta Maverick)

What it does

- Natively multimodal: processes image + text together
- Can answer open-ended questions about any image

Example output

- "The photo shows a 2020-era sedan with significant front-end damage: crumpled bumper, broken right headlight, and likely radiator damage. Estimated repair: \$4,500–6,000. Recommend total-loss review if vehicle value < \$8K."

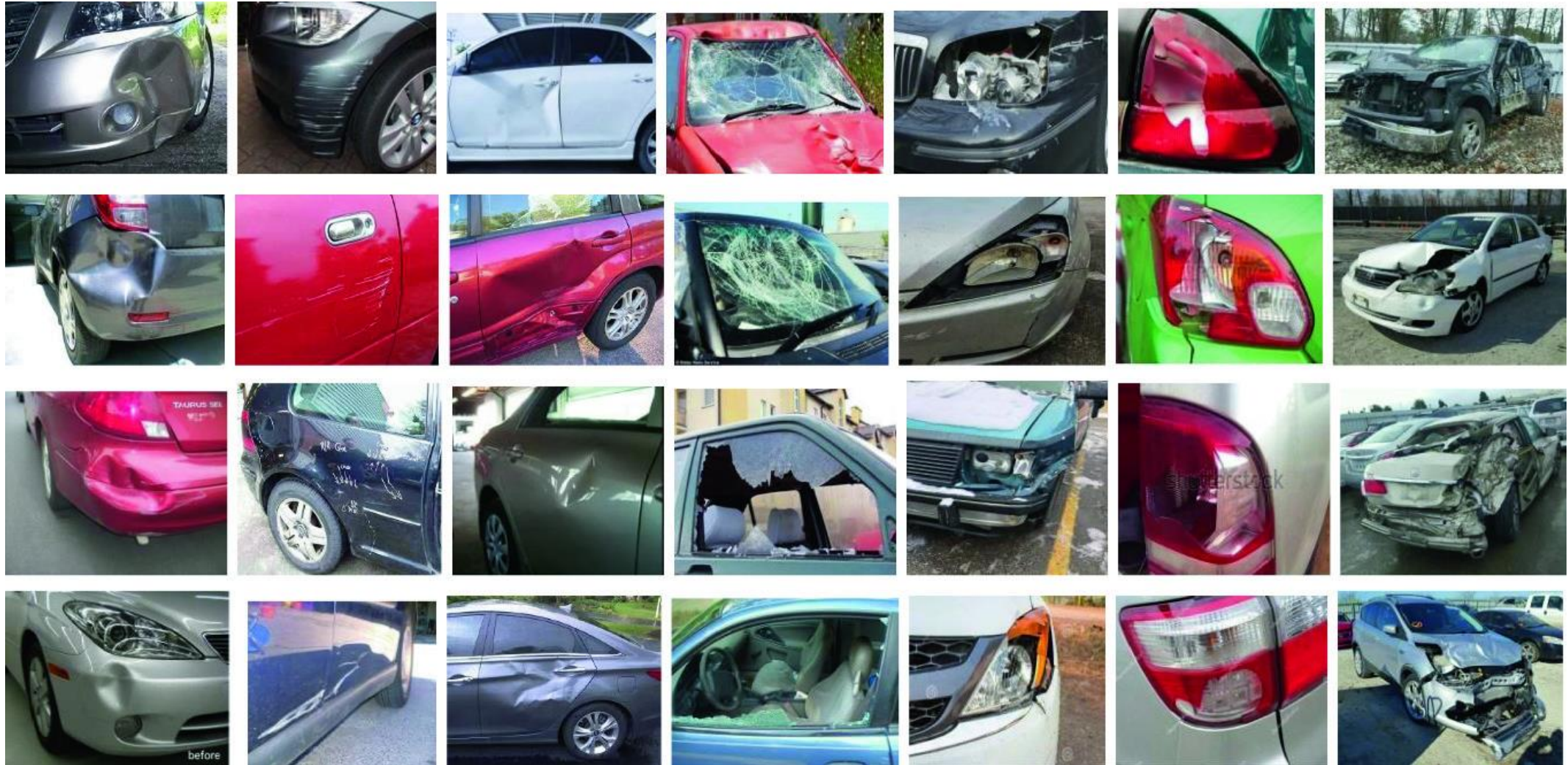
Strengths

- Reasons about context, generates explanations
- Zero-shot: no task-specific training needed
- One model handles many tasks via prompts

Limitations

- Larger, slower, more expensive to run
- Can hallucinate; less reliable for safety-critical decisions

Convolutional Neural Network – Image Recognition



Convolutional Neural Network – Retail

Use cases: shelf monitoring, planogram compliance, cashier-less checkout, theft detection



Shelf stock detection — Identify empty or misplaced products



Planogram compliance — Verify products match merchandising plan



Cashier-less checkout — Recognize items in cart (Amazon Go)



Shrinkage / theft detection — Flag suspicious behavior at self-checkout

Cashier-less Checkout – Retail



How a Vision Model Learns a 500 mL Coke Bottle

Teaching a computer to recognize one specific product

1

Collect

Take hundreds of photos of the 500 mL Coke from many angles, lights and backgrounds

2

Label

Draw a box around it and tag it “Coke 500 mL.” Also label look-alikes: cans, 1.25 L, Pepsi

3

Train

A neural network learns the tell-tale cues: red label, bottle shape, cap and proportions

4

Test and fix

Show it new photos; when it mixes up a can for the bottle, add examples and retrain

5

Deploy

Now it can spot a 500 mL Coke on the shelf or in a hand, in real time

The tricky part: judging size

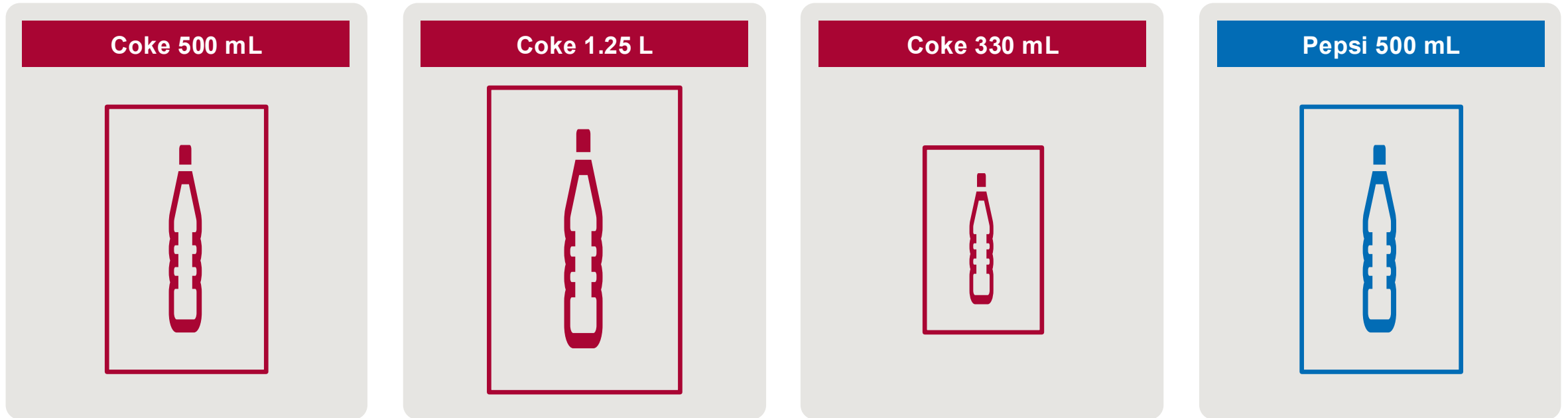


A flat photo does not show real size. The model learns size from context: how tall the bottle looks next to a shelf, a hand or nearby items.

That is why it needs many labeled examples of each size, so it can tell a 500 mL bottle from a 330 mL can or a 1.25 L bottle.

Labeling Examples: Teaching the Model Each Bottle

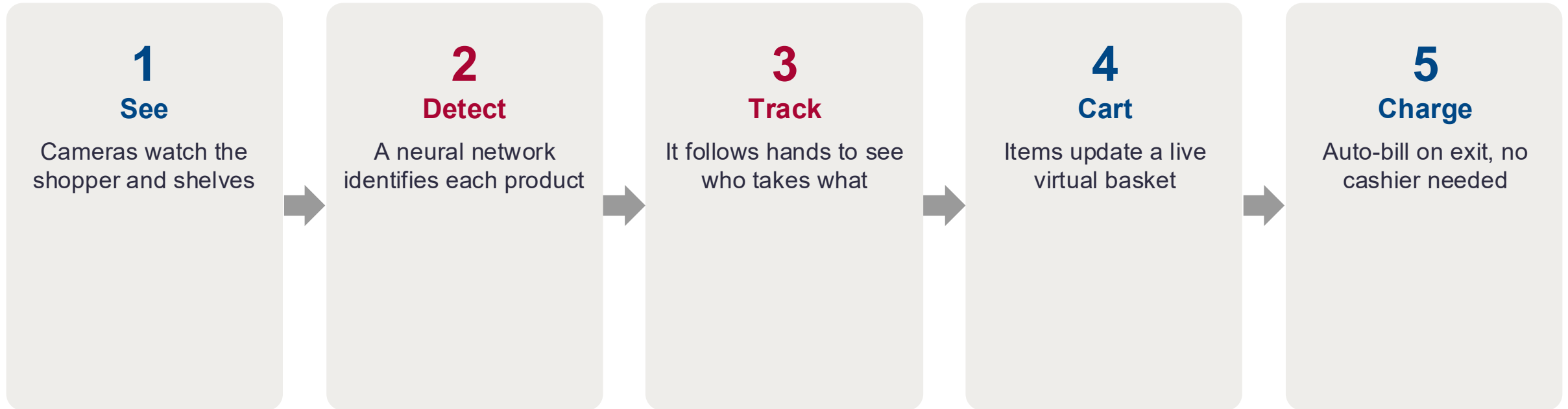
Every training photo gets a box and a precise tag



Annotators draw a box around each product and tag its exact brand and size. The model trains on thousands of these, including look-alikes like Pepsi and the same bottle from many angles, so it learns to tell them apart.

How Cashier-less Checkout Works

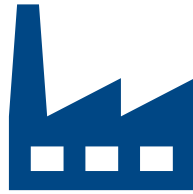
From camera to receipt in five steps



Red steps lean on AI (deep learning); **blue steps** lean on traditional software and sensors.

Convolutional Neural Network – Manufacturing

Use cases: defect detection, assembly verification, predictive maintenance, worker safety



Surface defect detection — Scratches, dents, weld flaws on production line



Assembly verification — Confirm all parts present & correctly placed

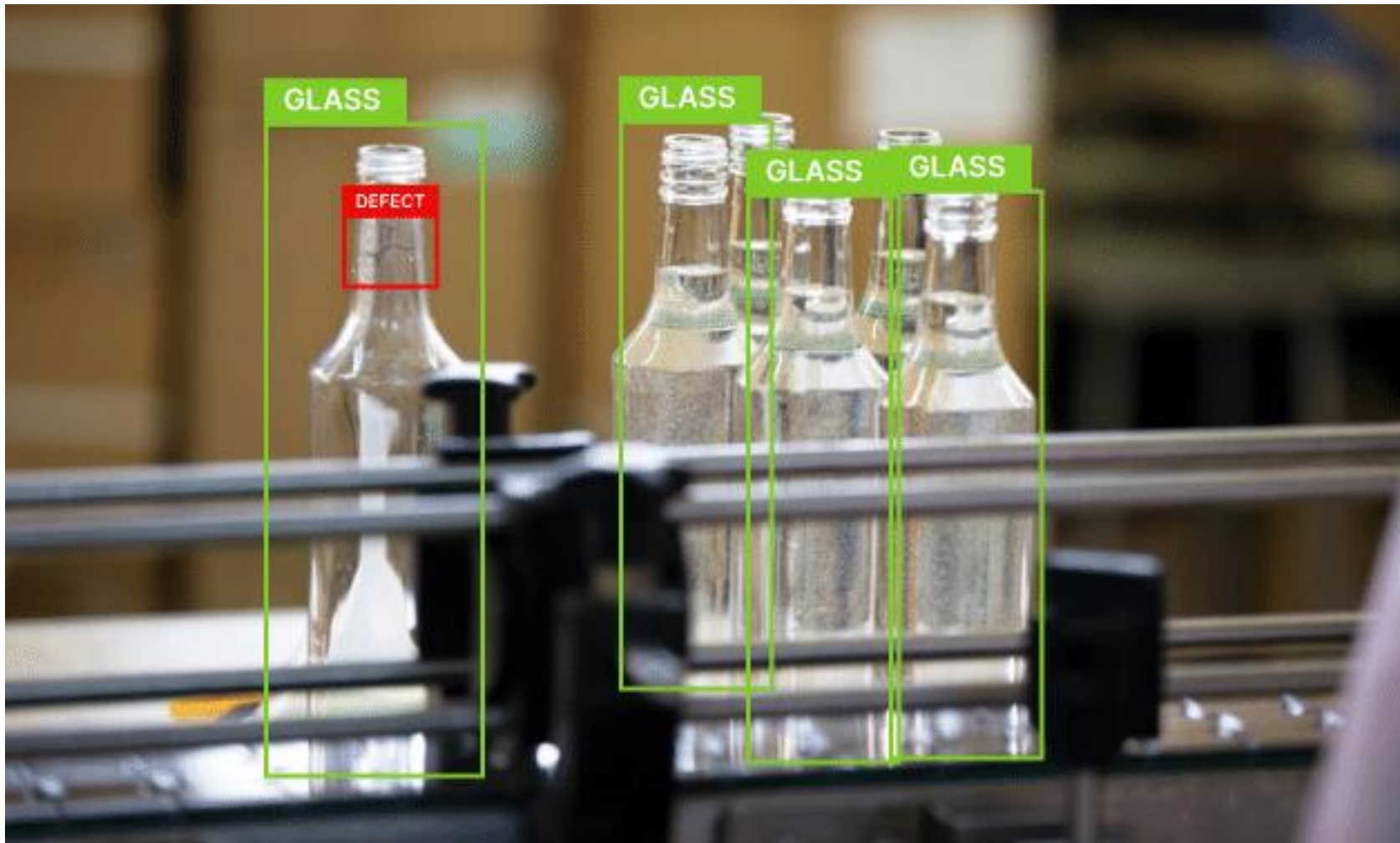


Predictive maintenance — Spot wear, corrosion, leaks via thermal/visual



Worker safety / PPE — Detect missing hard hats, vests, safety zones

Scratch Detection – Manufacturing

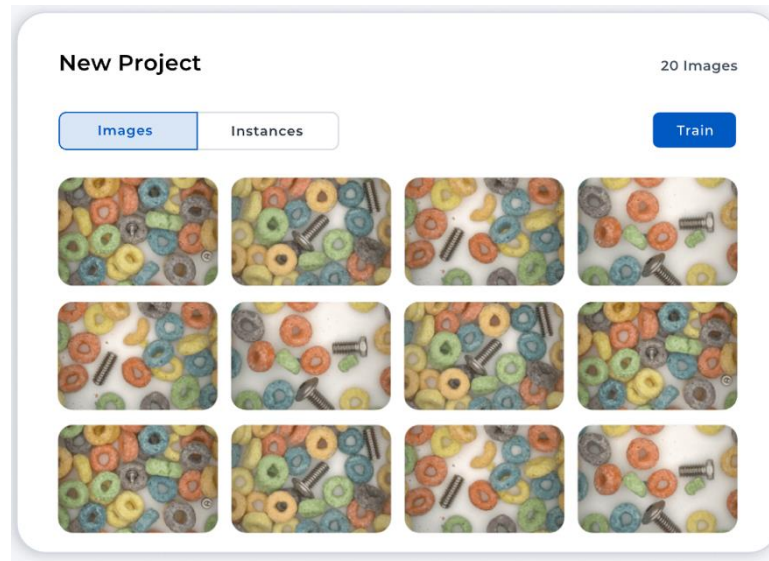


Convolutional Neural Network – Image Recognition

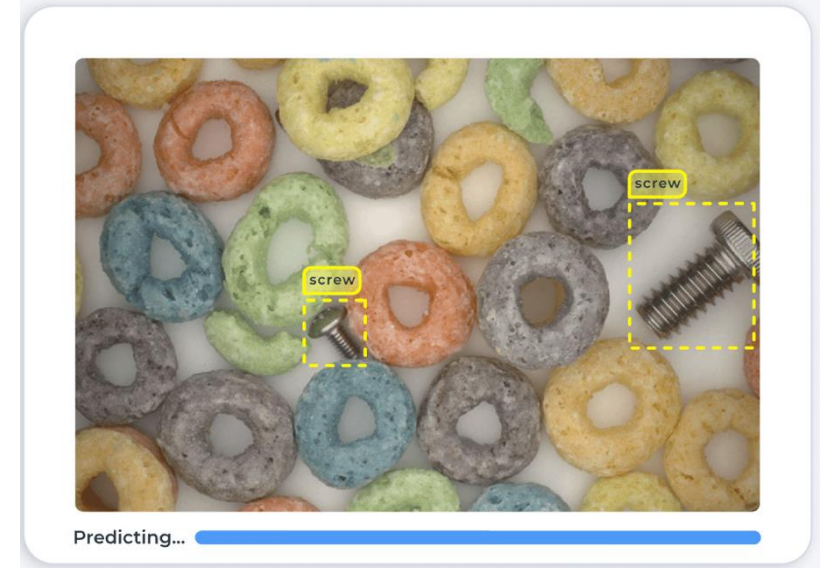
1. Label



2. Train

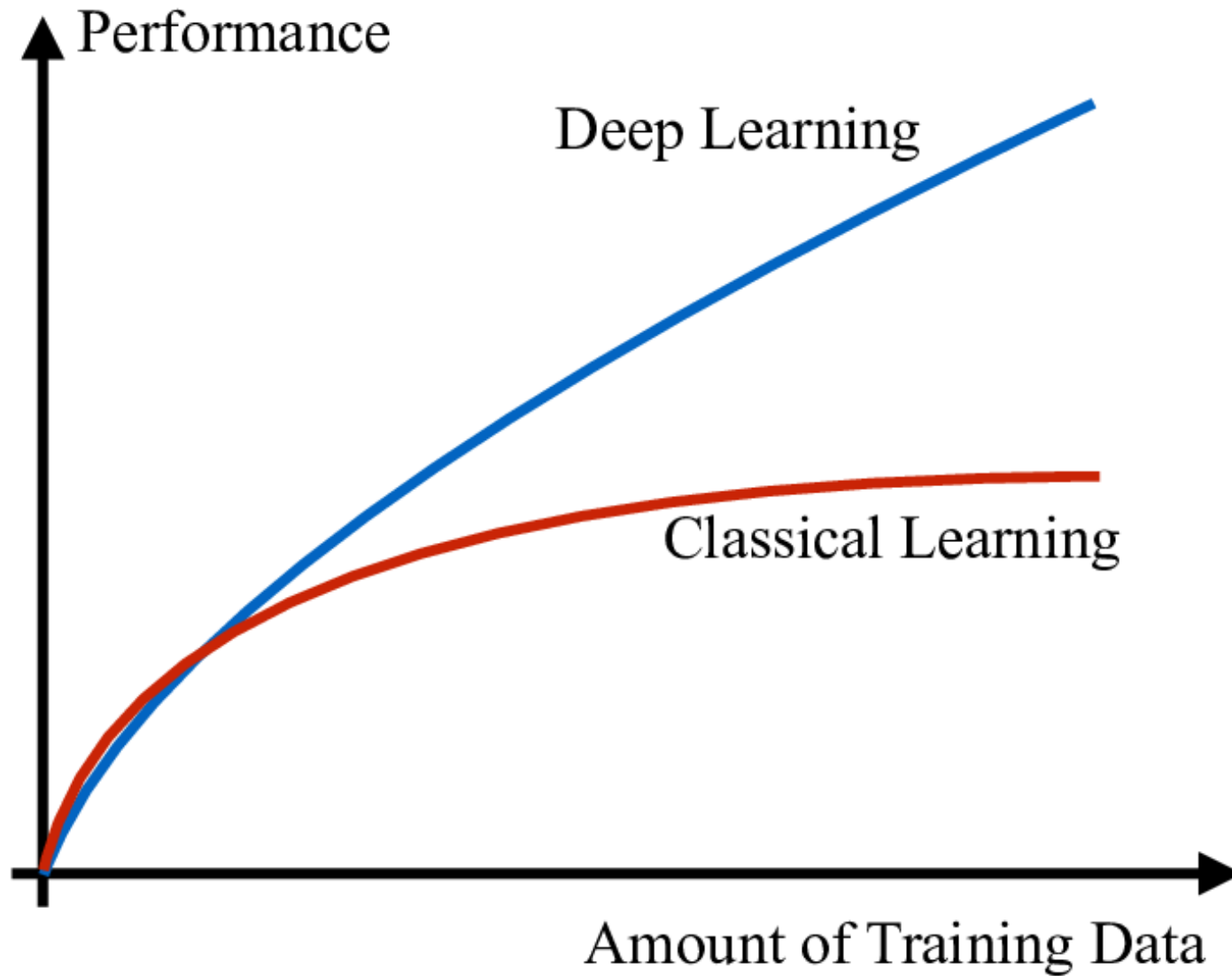


3. Deploy



Source <https://landing.ai/landinglens>

Video 3: Train, Validate, Test



Deep Learning Frameworks



TensorFlow

TensorFlow

Google's industry-standard framework. Production-ready deployment via TF Serving and TFLite. Includes Keras as its high-level API.



Keras



PyTorch

Meta's research-favorite framework. Dynamic computation graphs and Pythonic API. Powers most modern LLMs and academic papers.



Hugging Face

Hub of pre-trained models and the Transformers library. De facto standard for working with LLMs and foundation models.

What are some uses for how you would like to incorporate deep learning?



Computer Vision

Image recognition, object detection



Natural Language

Translation, summarization, chatbots



Speech & Audio

Voice assistants, transcription



Fraud Detection

Anomaly detection in transactions



Healthcare

Diagnosis, drug discovery, imaging



Autonomous Systems

Self-driving, robotics, drones



Recommendations

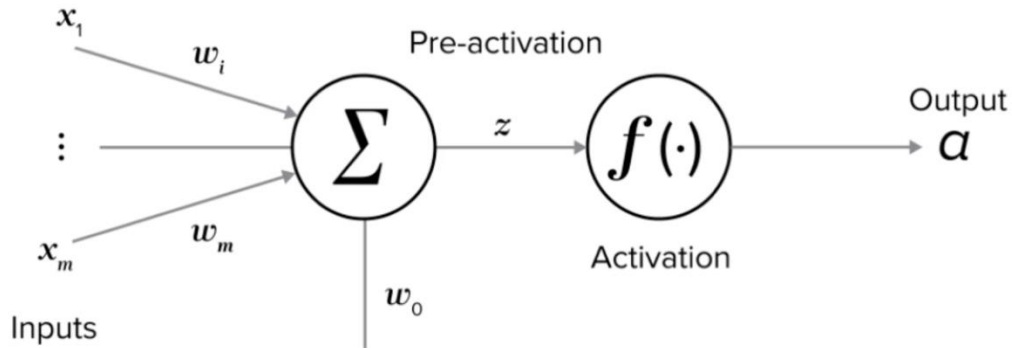
Netflix, Amazon, ad targeting



Forecasting

Demand, pricing, financial models

Neural Networks



Inputs ($x_1 \dots x_m$)

Features fed into the neuron — pixels, words, sensor readings, anything numeric.

Weights ($w_1 \dots w_m$) + bias (w_0)

Each input is multiplied by a learned weight. The bias shifts the result up or down.

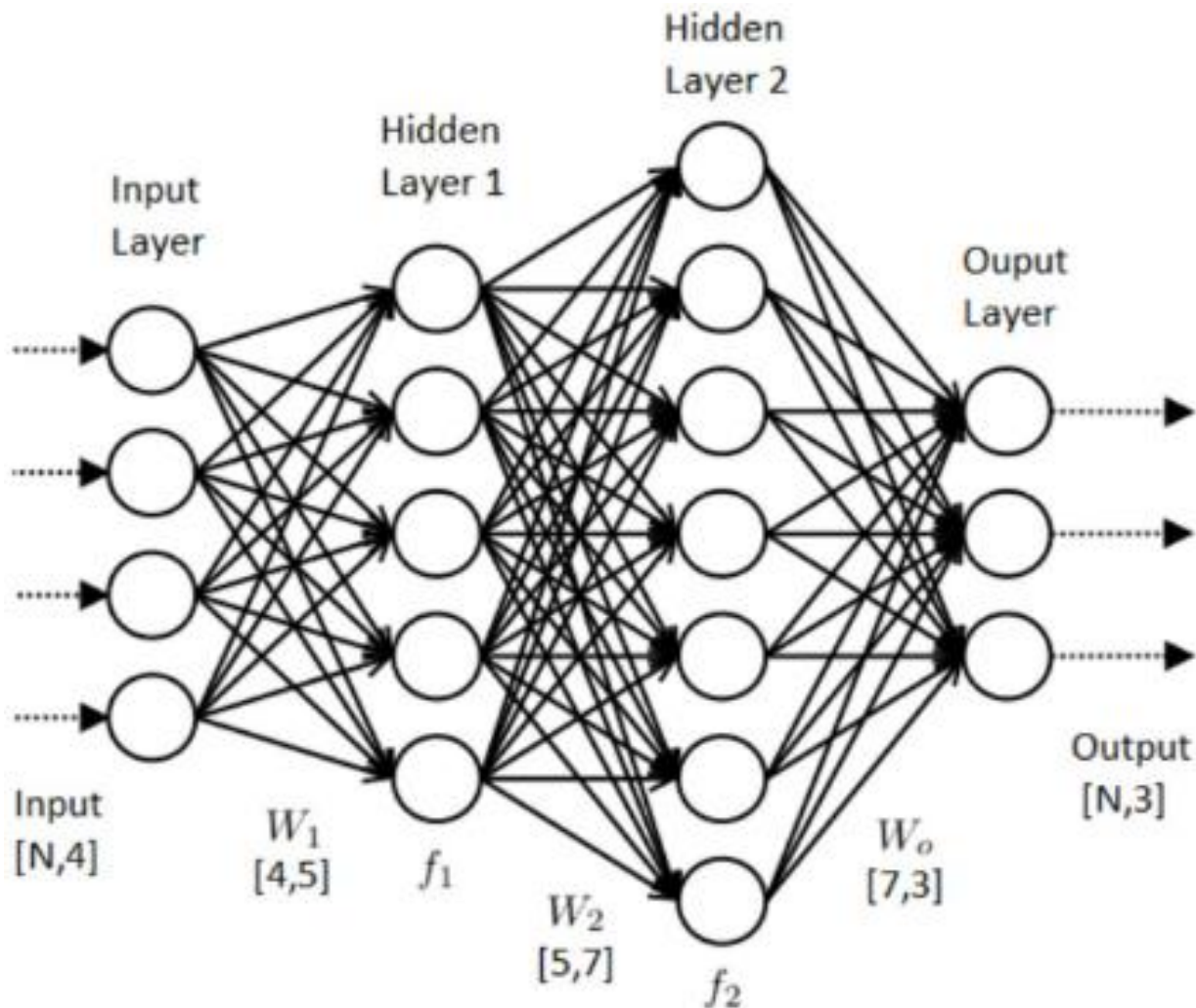
Pre-activation (z)

Weighted sum: $z = \sum w_i \cdot x_i + w_0$. A single number summarizing the inputs.

Activation $f(\cdot) \rightarrow$ output a

A nonlinear function (sigmoid, ReLU, tanh) maps z to the final output. This nonlinearity is what lets neural networks learn complex patterns.

Deep Learning



Input Layer

Receives raw features (here, 4 inputs producing a batch of size N).

Hidden Layers

Stacks of neurons that learn intermediate representations. Each layer transforms its input through weights (W_1, W_2) and an activation (f_1, f_2). Adding layers = more abstract features.

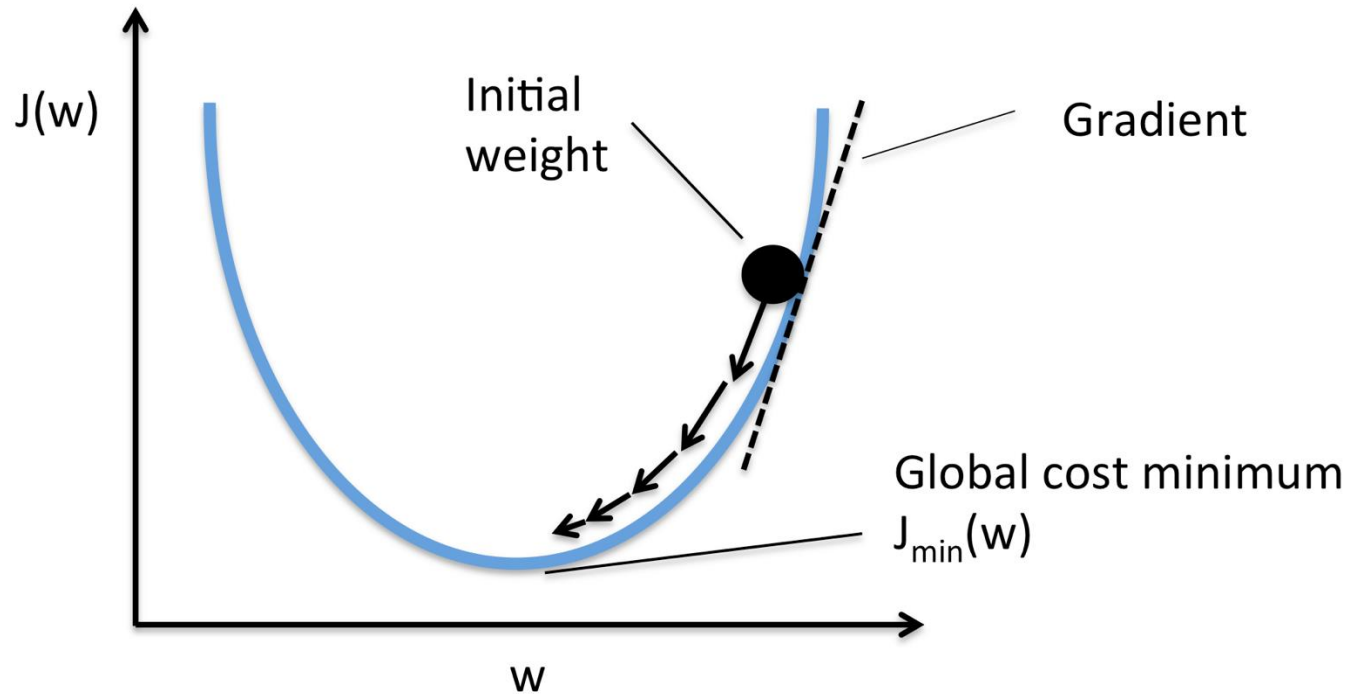
Output Layer

Produces the final prediction (here, 3 outputs per example). Shape depends on the task: 1 number for regression, k numbers for k -class classification.

Why "deep"?

Two or more hidden layers makes a network "deep." Depth lets the model compose simple patterns into complex ones.

Gradient Descent



Example: Predicting a home worth \$400K, the model guesses \$1M (way off), then \$620K, then \$450K, then \$405K. Each round the error shrinks, so the model gets a little smarter.

The goal: make fewer mistakes

Picture the curve as a valley. High up, the model gets a lot wrong. The bottom is where it makes the fewest mistakes, and that is where we want to end up.

How it works

Start somewhere on the hillside. Look at which way is downhill, take a small step that direction, and repeat until you cannot go any lower.

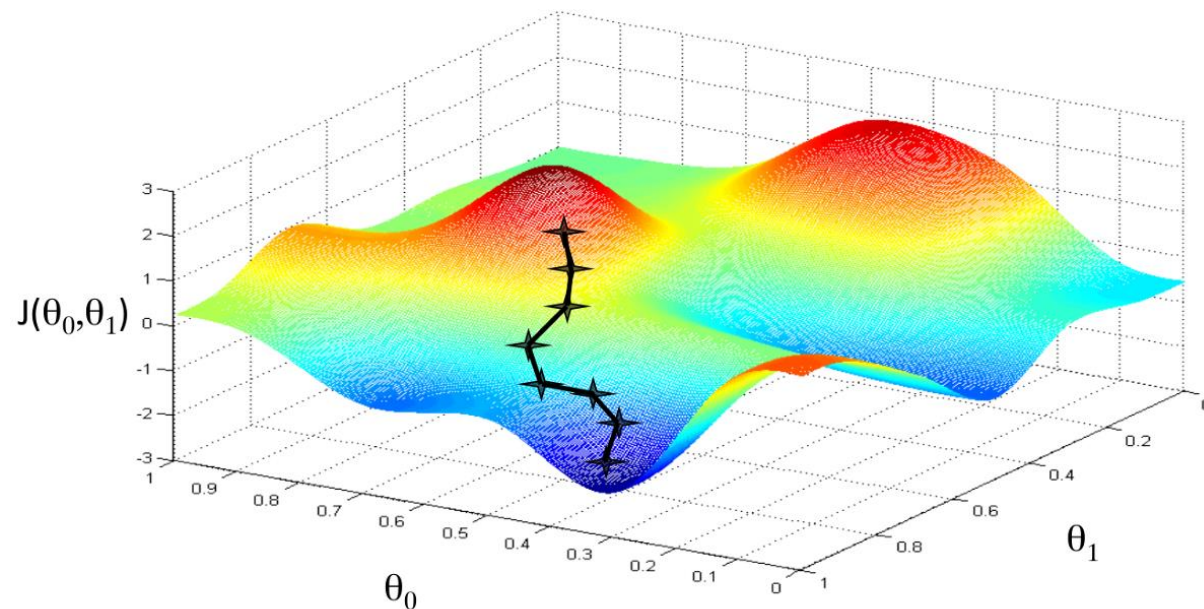
The everyday picture

It is like a ball rolling into a valley. Gravity pulls it downhill until it settles at the lowest point, the best answer.

Where it fits in machine learning

Training a model means tuning its internal settings so its predictions get better. Gradient descent is the method that does that tuning, and this downhill nudging is exactly what we call "learning."

Gradient Descent



What it's really doing

The model is hunting for the settings that give the best answers, the lowest point in the valley.

One step at a time

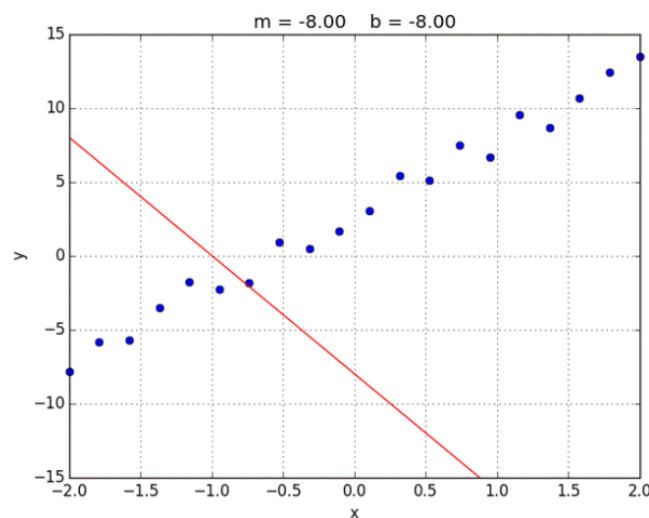
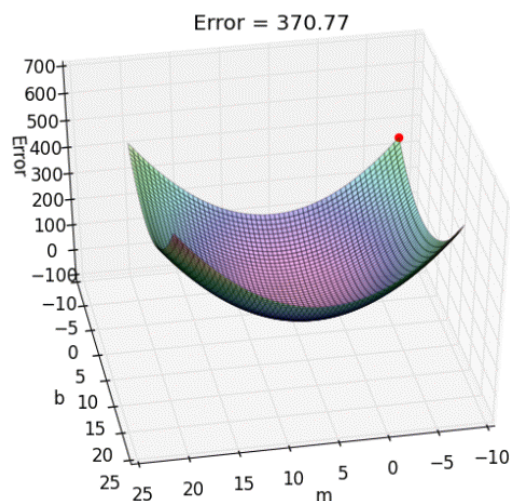
At each spot it checks which way is downhill, takes a small step that direction, and repeats thousands of times.

How big a step?

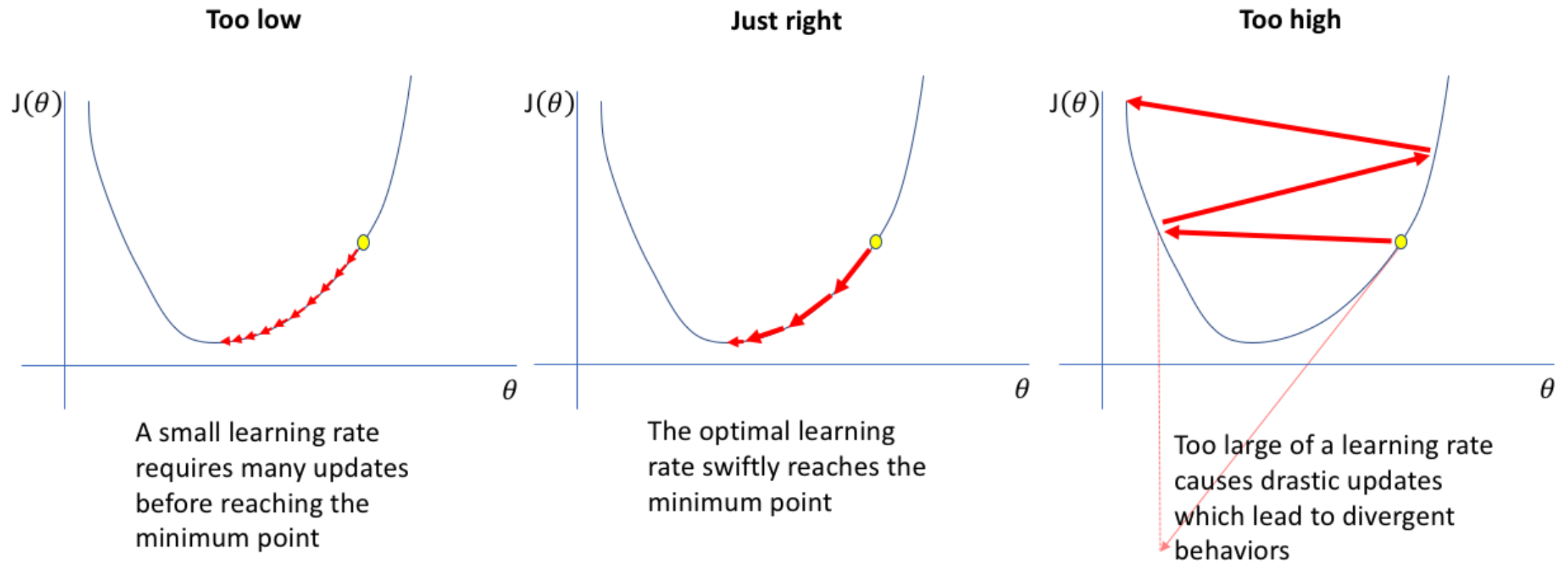
Steps too big and it overshoots the bottom. Too small and it takes forever. Getting the step size right is the key.

Where it fits in machine learning

This is how models actually learn. Every training run uses gradient descent to adjust millions of settings until the predictions are as accurate as possible.



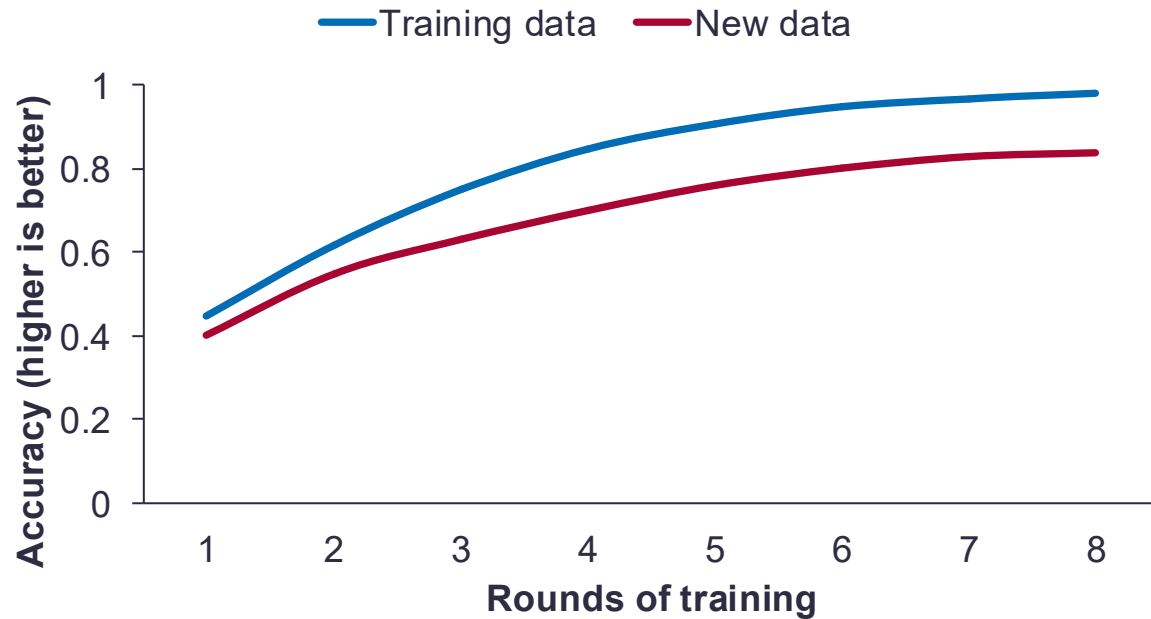
Learning Rate



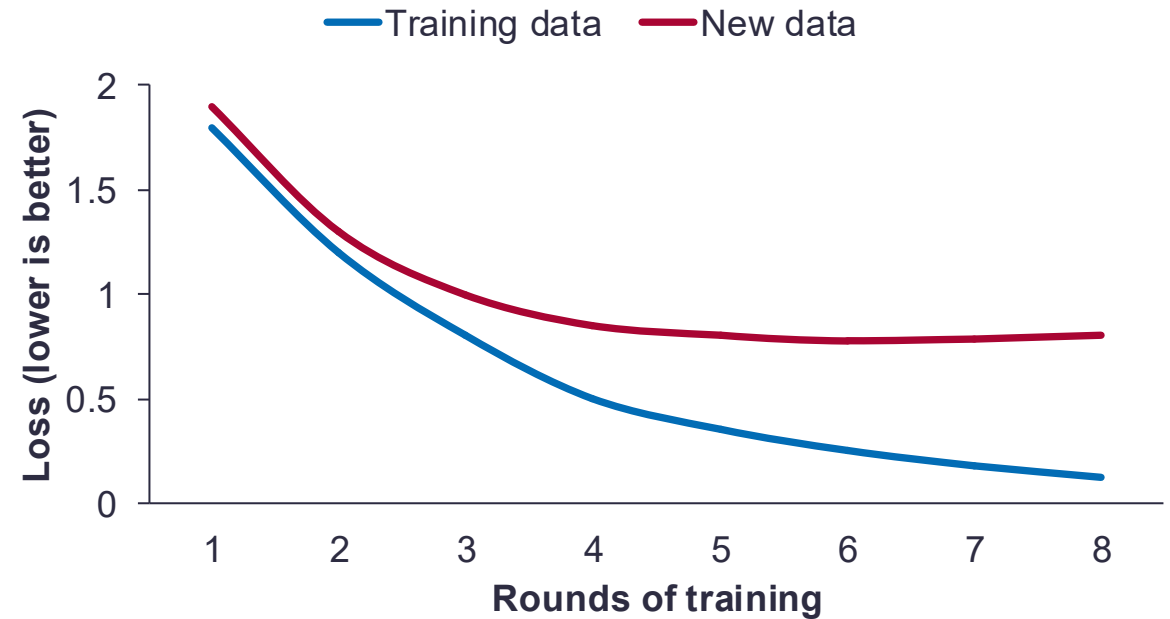
Key takeaway: The learning rate is simply how big a step the model takes each time it learns. Too big and it bounces around and never settles; too small and training drags on forever. The trick is finding a step size that's just right.

Plotting Loss and Accuracy

Accuracy climbs, then levels off



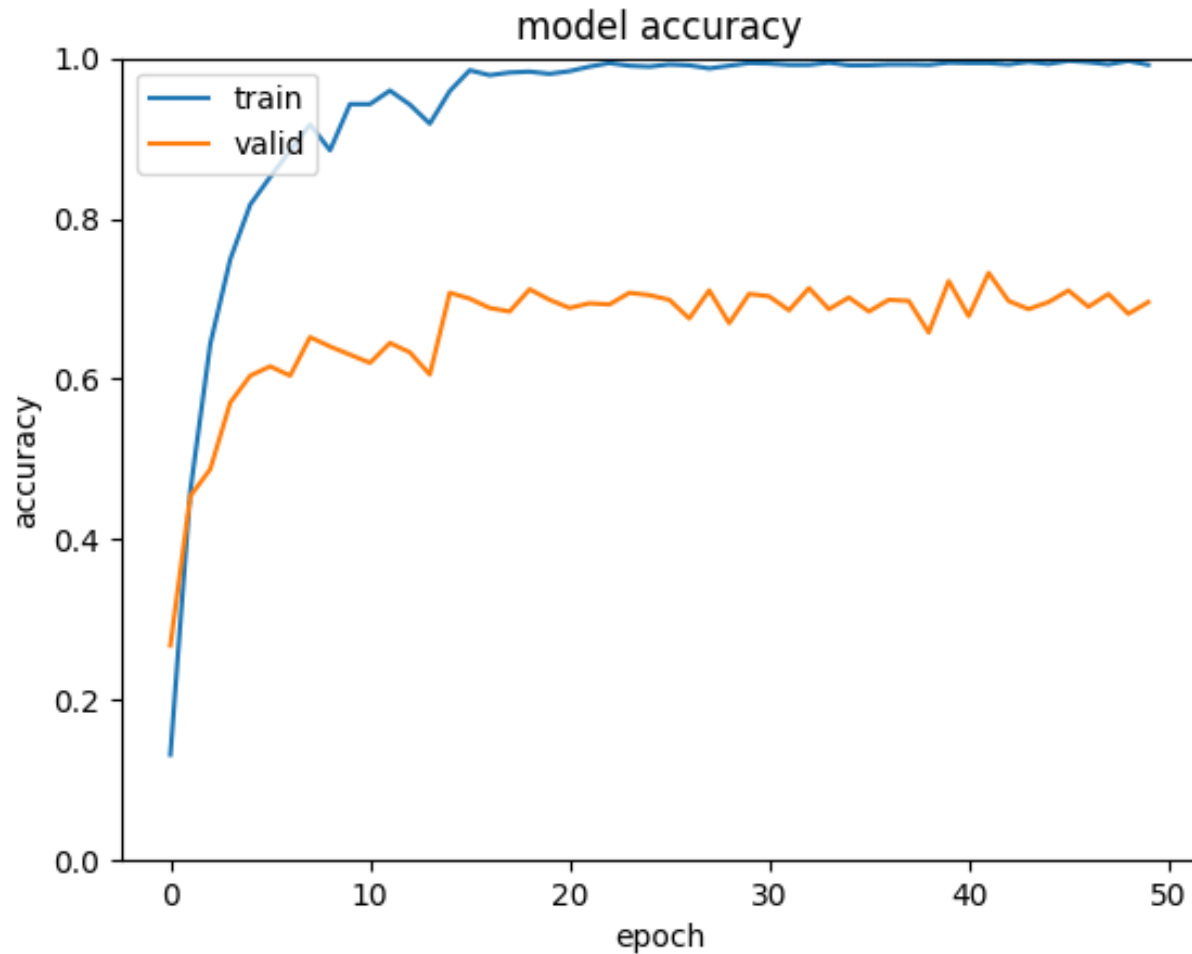
Loss falls, then levels off



How to read these

Each chart follows the model as it trains. Blue is the data it practices on; red is fresh data it has never seen. Accuracy should climb toward the top, and loss (its mistakes) should fall toward the bottom. When both lines flatten out, the model has learned about as much as it can.

How to Know When Learning Has Stopped



The lines flatten out

Early on, both lines climb fast: the model is learning quickly. After a while they go flat and stop rising.

Learning has stopped

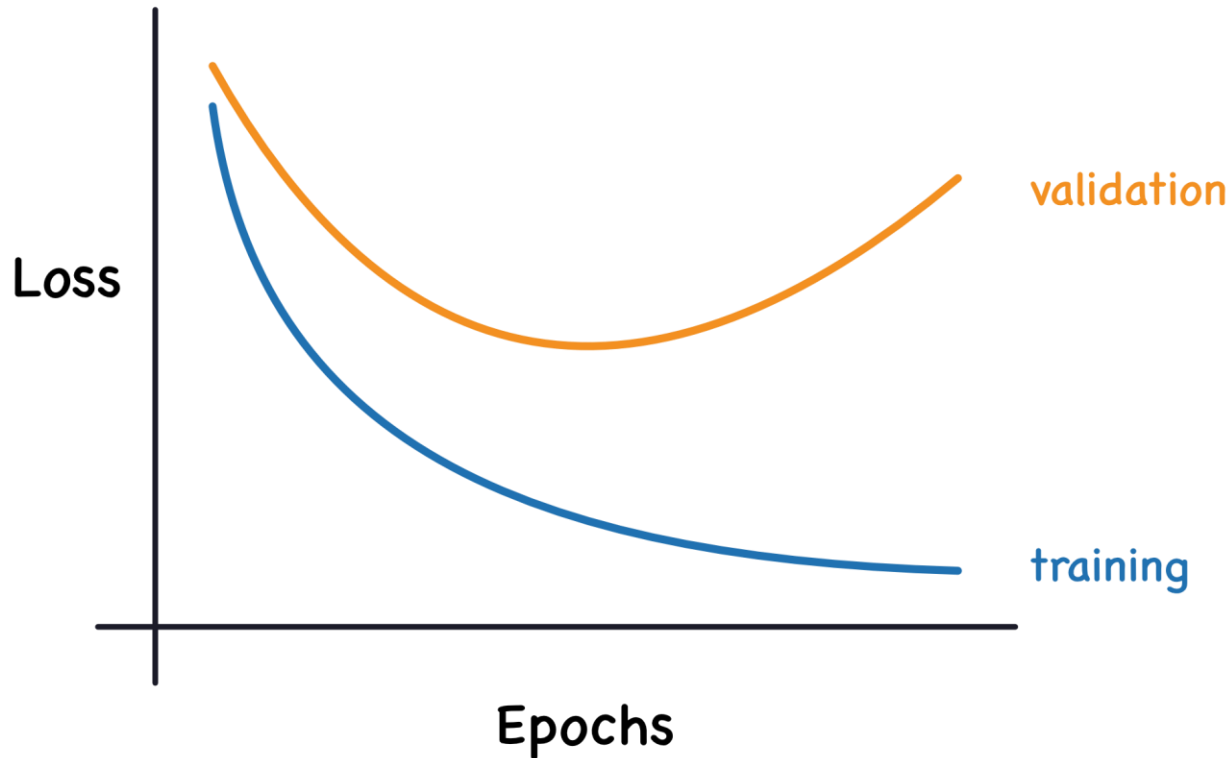
Extra training rounds no longer help. The model has gotten about all it can from the data.

What to do

This is the signal to stop training (called “early stopping”). It saves time and stops the model from just memorizing the practice data.

Overfitting

The Learning Curves



Two lines to watch

Blue is the practice data; orange is new data the model has never seen.

Where it goes wrong

The blue line keeps improving, but orange dips and then rises again. The model gets better on practice questions while getting worse on real ones.

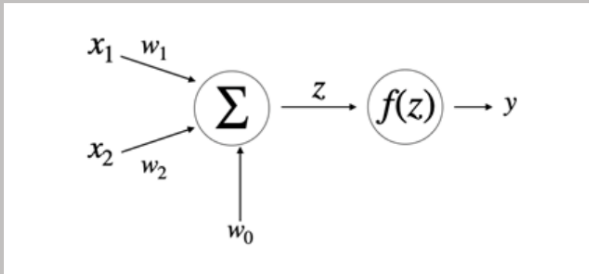
That is overfitting

It has started memorizing the practice data instead of learning the general pattern, like a student who memorizes the answer key but fails the real exam.

The fix

Stop training at the low point of the orange line and give the model more varied data.

Quiz 3.1



You want to train this network with respect to a loss function defined by:

$$J = (\hat{y} - y)^2 = \epsilon^2$$

Assume then that the network has a learning rate equal to 0.1. The training samples are given by:

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

and, $y = 6$.

Finally, assume that the weights are given by:

$$w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

What are the gradients?

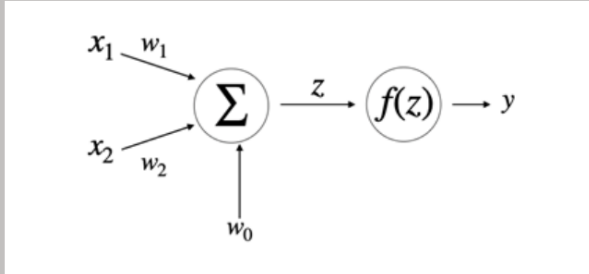
Let's start with what is the predicted output.

1. $\hat{y} = 1$ (initial weights are 1)
2. $\hat{y} = 1 + 1 * 1 + 1 * 1 = 3$
3. $\epsilon = (\hat{y} - y) = (3 - 6) = -3$

1. Note should be $J = \epsilon^2$

Quiz 3.1

An image of the network is given below:



You want to train this network with respect to a loss function defined by:

$$J = (\hat{y} - y)^2 = \epsilon$$

Assume then that the network has a learning rate equal to 0.1. The training samples are given by:

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

and, $y = 6$.

Finally, assume that the weights are given by:

$$w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

What are the gradients?

1. $\nabla J w_0 = 2\epsilon x_2 = -6$
2. $\nabla J w_1 = 2\epsilon x_2 = -6$
3. $\nabla J w_2 = 2\epsilon x_2 = -6$